

Aufgabe 1: Transformers

Teilnahme-ID: 79418

Bearbeiter/-in dieser Aufgabe:
Fabian Schwarz

April 6, 2026

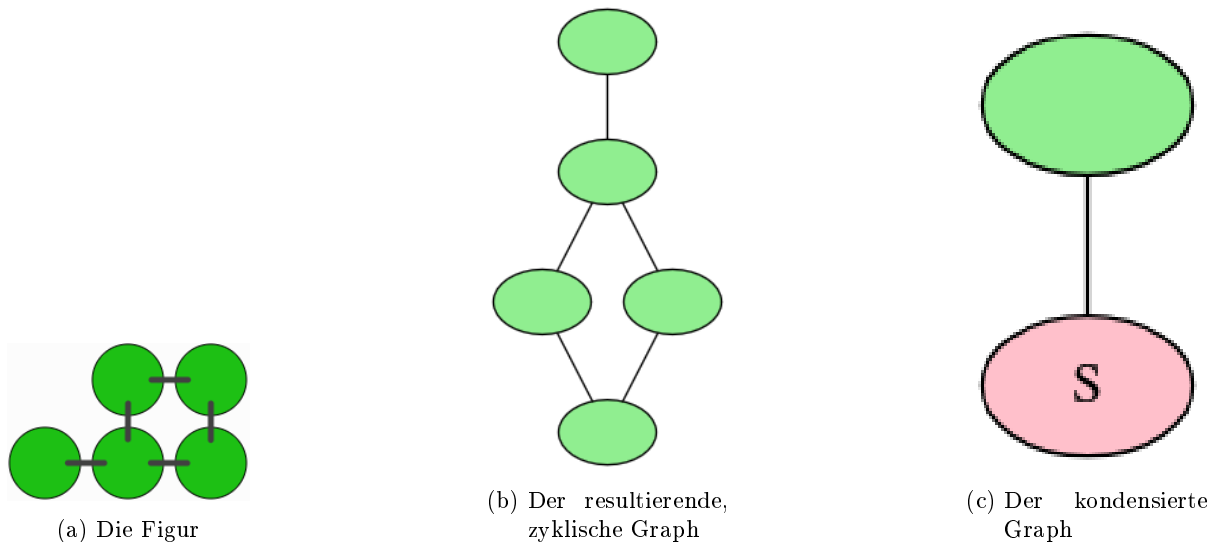
Contents

1	Lösungsidee	1
2	Umsetzung	3
3	Laufzeit	5
4	Werkzeuge	5
5	Beispiele	5
6	Quellcode	5

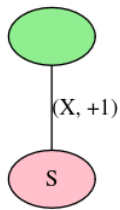
1 Lösungsidee

Man modelliert jede Figur als einen ungerichteten, zusammenhängenden Graphen $G = (V, E)$. Die Knotenmenge V besteht hier aus den Steckplättchen, und die Kantenmenge E aus den Stiften. Jeder Stift $e \in E$ ist hier aus einer Achse (horizontal bzw. vertikal) und einer Richtung (-1 bzw. +1) zusammengesetzt.

Man bemerkt, dass Stifte, die Teil eines Zyklus sind, nicht gedreht werden können, da die Figur sonst "zerreißt". Man kondensiert den Graphen also zu einem neuen Graphen G' , in dem alle Knoten, die durch Zyklen verbunden sind, zu einem Superknoten (S) zusammengefasst sind. Dies sieht beispielsweise so aus:

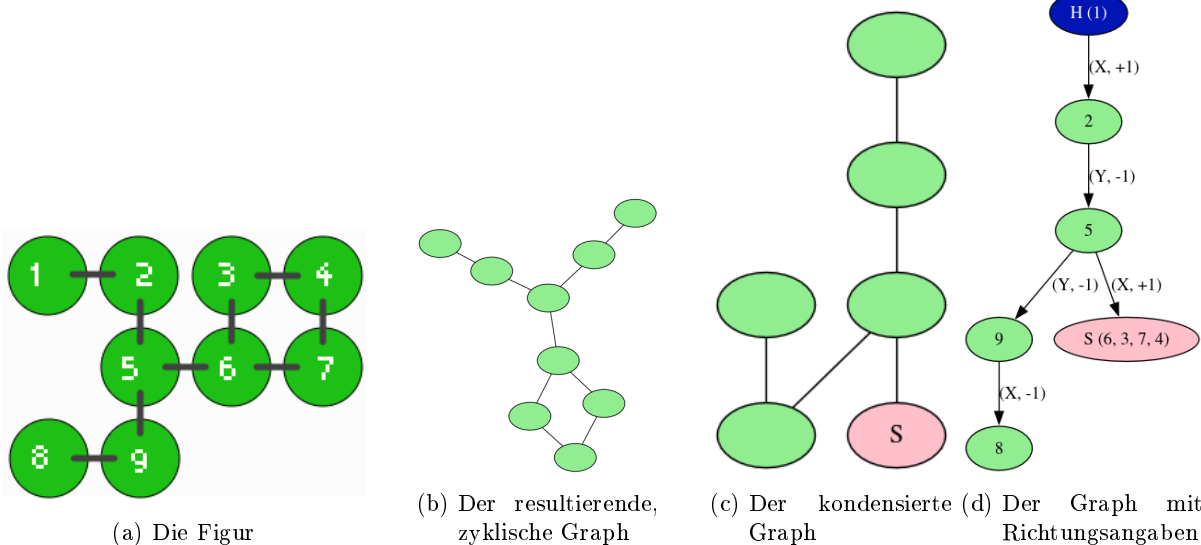


Schließlich markiert man die Stifte mit ihren entsprechenden Richtungen:



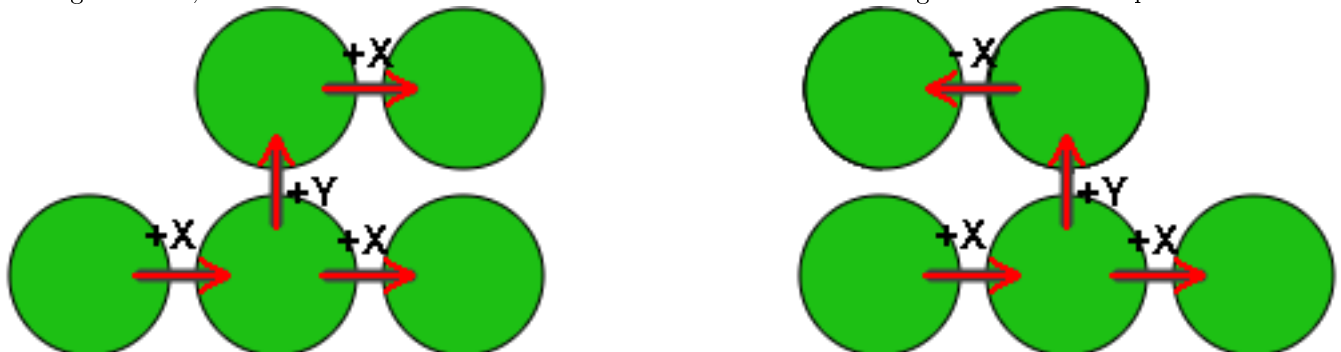
Der Stift ist horizontal, daher X , und verläuft von dem Ursprungsknoten aus nach rechts, daher $+1$. Man stellt hier fest, dass nach Beseitigung aller Zyklen der Graph, da er ja bereits zusammenhängend ist, einen Baum darstellt.

Der Ursprungsknoten muss arbiträr, aber eindeutig, ausgewählt werden. Man hat aktuell noch keinerlei Wissen darüber, welcher Knoten welchem Gegenknoten im anderen Graphen entspricht. Es macht daher Sinn, die Zentren des Baums zu verwenden, wie sich in der Umsetzung zeigt. Ein komplexeres Beispiel könnte wie folgt aussehen:



Zu Beginn werden den Plättchen beliebige Indices verliehen. Diese bleiben bei der Umwandlung in den Graphen natürlich erhalten, sind nur hier nicht abgebildet. Es werden erneut Zyklen zu Superblöcken kondensiert, und schließlich ein hier noch arbiträrer Ursprungsknoten (H) gewählt, von welchem aus der Graph traversiert und die Verbindungen beschriftet werden. Der Graph bleibt ungerichtet, ist hier aber, um das Traversieren zu kennzeichnen, mit Pfeilen dargestellt. Im Superknoten ist gespeichert, aus welchen Knoten dieser zusammengesetzt ist.

Bei einer 180° -Drehung um einen Stift beobachtet man, dass alle Stifte parallel zur Drehachse ihre Richtung behalten, während alle senkrechten Stifte das Vorzeichen ihrer Richtung ändern. Ein Beispiel:



Hier wird um den $+Y$ -Stift gedreht.

Aus diesen Bedingungen lassen sich also Regeln für die Überführbarkeit zweier Figuren in einander anhand ihrer kondensierten Bäume bestimmen:

- Die Bäume A und B müssen isomorph sein, das heißt, die gleiche Anzahl an Knoten und Kanten haben, sowie Kanten auf den gleichen Achsen zwischen entsprechenden Knoten. Besteht der

eine Baum also ausgehend von seiner Wurzel aus $V_1 = (+X, +Y, -X)$, so muss V_2 auch eine $(\pm X, \pm Y, \pm X)$ Abfolge sein.

- Zyklen müssen in beiden Figuren identisch sein, d.h., die gleiche Länge und die gleiche Abfolge von X- und Y-Verbindungen besitzen; die genaue Richtung (± 1) ist erstmal irrelevant.
- In Baum B muss es pro Drehung genau ein Plättchen P geben, dass als Drehpunkt fungiert. An ihm ist eine Drehachse X oder Y definiert. Alle Stifte, die über diese Drehachse an P hängen, behalten ihre Richtung bei. Alle Stifte, die über die andere Achse an P hängen, folgen den oben festgestellten Regeln: laufen sie parallel zur Drehachse, behalten sie ihre Richtung identisch bei; andernfalls ändert sich das Vorzeichen der Richtung.
- Zyklen müssen entweder vollständig identisch bleiben, d.h., identische Abfolgen an Verbindungen besitzen, oder, wenn sie gedreht wurden, nach der oben genannten Inversionsregel übereinstimmen. Beispielsweise kann $(+Y, +X, -Y, -X)$ nur identisch bleiben, zu $(-Y, +X, +Y, -X)$ oder zu $(+Y, -X, -Y, +X)$ werden. Diese Abfolgen haben natürlich keinen festen Startpunkt, d.h. $(+Y, +X, -Y, -X) = (-X, +Y, +X, -Y)$.

2 Umsetzung

Zu Beginn werden die Figuren eingelesen und als ungerichtete, gewichtete Graphen dargestellt. Die Gewichte sind hier die Richtung, bestehen also aus $\text{axis} \in X, Y$ und $\text{sign} \in -1, +1$.

Um nun Zyklen zu erkennen, werden die Graphen mithilfe einer Tiefensuche in zweifach zusammenhängende Komponenten (*Biconnected Components* nach Hopcroft and Tarjan (1973)¹) geteilt. Artikulationspunkte sind mögliche Drehpunkte unserer Figur, die resultierenden “Blöcke” die starren Zyklen. Hierfür werden während der DFS alle Kanten auf einen Stack gelegt und, sobald ein Artikulationspunkt gefunden wird, alle Plättchen bis zu diesem Punkt als starrer Block betrachtet.

Ein Artikulationspunkt ermittelt sich wie folgt:

1. Man traversiere den Graphen wie gewöhnlich in einer DFS, wobei alle traversierten Kanten auf einen Stack gelegt werden.
2. Jedem Knoten wird eine Entdeckungszeit (en. *discovery time*) `disc` zugewiesen, die für jeden Ablaufschritt inkrementiert wird. Ein zweites Feld `low` wird ebenfalls auf die Entdeckungszeit gesetzt.
3. Während jeder Backtracking-Phase betrachte man:
 - a) für jeden Knoten u , rückwärts gehend, all seine (bereits besuchten) Nachbarn v_i , bis auf seinen direkten Vorgänger. Die von allen Nachbarn v_i geringste `disc` übernimmt der Knoten u als `low`.
 - b) für jede Kante $u \rightarrow v$, rückwärts gehend², ob $v.\text{low} \geq u.\text{disc}$. Falls eine Gleichheit vorliegt, handelt es sich um einen Artikulationspunkt. Alle Knoten, die seit Beginn des backward-pass abgelaufen wurden, werden in den Artikulationspunkt als Superknoten vereint.³ Wenn der `low`-Wert echt größer ist, ist diese Verbindung eine Brücke, d.h. ein drehbarer Stift, und wird im resultierenden Baum zu einer Kante zwischen zwei (Super-)Knoten.

Hiermit zerfällt die Figur in eine Menge von Superknoten, Artikulationspunkten (gewöhnlichen Plättchen) und Brücken, die sich zu dem kondensierten Baum zusammenfassen lassen (dem sog. *block-cut tree*). Ein solcher Baum behält nun also folgende Informationen:

- Jeder Superknoten beinhaltet seine interne, zyklische Abfolge an Verbindungen, z.B. $(+X, +Y, -X, -Y)$, sowie seine Verbindungen nach außen
- Jedes Plättchen, das kein Teil eines Zyklus ist, bleibt ein Knoten

¹Hopcroft, J., Tarjan, R. (1973). Algorithm 447: efficient algorithms for graph manipulation. *Communications of the ACM*, 16(6), 372–378. <https://doi.org/10.1145/362248.362272>

²Im forward-pass wurde also eine Kante $u \rightarrow v$ abgelaufen; eigentlich läuft man hier von $v \rightarrow u$ rückwärts, dennoch wird die Kante hier als $u \rightarrow v$ beschrieben, wobei u die “ältere” ist.

³Der Artikulationspunkt ist der Punkt, der den Block mit dem Rest des Graphen verbindet. Die Wurzel der DFS ist nur dann ein Artikulationspunkt, wenn sie im DFS-Baum mehr als ein Kind hat.

- Die Brücken, also alle Kanten außerhalb von Zyklen, werden zu den Kanten des Baums

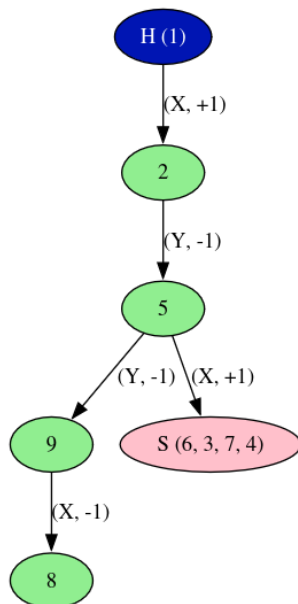
Nun erfolgen die ersten Überprüfungen:

1. Haben beide Bäume die gleiche Anzahl an Knoten und Kanten?
2. Sind die (ungeordneten) Listen der Grad-Zahlen der Knoten identisch?

Ist eine der Überprüfungen falsch, sind die Figuren nicht in einander überführbar.

Um nun zwei Bäume zu vergleichen, müssen sie in eine Form gebracht werden, in der die genaue Indizierung der Knoten für Vergleiche irrelevant wird. Hierzu werden von beiden Bäumen sukzessiv alle Blätter entfernt, bis nur noch 1 oder 2 Knoten übrig bleiben. Diese Punkte werden Zentren genannt, die folgend als Wurzeln der Bäume verwendet werden. Hat einer der Bäume zwei Zentren, der andere jedoch nur eines, können die Bäume nicht isomorph sein. Haben beide Bäume zwei Zentren, wird das erste Zentrum von Baum A mit beiden Zentren des Baums B Gegengeprüft, und damit folgende Schritte zwei mal durchgeführt. Nur wenn beide scheitert, scheitert die ganze Überprüfung.

Von den Wurzeln R_A und R_B aus wird nun der Baum traversiert, und eine Signatur erstellt. Diese setzt sich aus einer beliebig, aber immer gleich, sortierten Liste der Signaturen der Kinder und Achsen zu ihnen zusammen. Ein Beispiel:



Hier ermittelt sich Knoten 5 als Zentrum. Also wird der Baum, entgegen der in der Lösungsidee gewählten Richtung, von ihm aus traversiert: Er hat die Kinder 2, 9, und $S(6,3,7,4)$. Es wird sich für eine Sortierung nach Index entschieden, also ergibt sich (vorerst) die Signatur: $[(+Y, 2), (-Y, 9), (+X, S(\dots))]$. Die Knoten werden nun erneut nach ihren Signaturen aufgelöst. Knoten 2 hat Knoten 1 als Kind, der ein Blatt ist. Knoten 9 hat 8 als Kind, ebenfalls ein Blatt. Der Superknoten S hat intern seine Abfolge von Steckachsen gespeichert, also wird diese als seine Signatur eingefügt. Daher erweitert sich die Signatur insgesamt zu $[(+Y, [(-X, "Blatt")]), (-Y, [(-X, "Blatt")]), (+X, [S(+Y, +X, -Y, -X);])]$. Die gewählte Signatur von S ist hier beispielhaft ein einfacher Zyklus aus 4 Knoten; die Vorzeichen entsprechen nicht genau dem Beispiel von oben. Nach dem Semikolon würde wie gewohnt eine Liste der Nachbarknoten des Superknotens folgen, wenn er welche hätte. Diese Sonderregel ist wichtig, da sonst später die Übereinstimmung der Zyklen nicht geprüft werden kann. Es ist zu beachten, dass die Achsenabfolgen innerhalb von Superknoten zyklisch sind, d.h. weiterhin $(+Y, +X, -Y, -X) = (-X, +Y, +X, -Y)$, und so weiter. Auch Zyklen, die die Abfolge genau "rückwärts" haben, sind identisch.

Wenn die Signaturen zweier Bäume nun übereinstimmen, sind sie garantiert isomorph. Hierbei dürfen die Vorzeichen der Verbindungen abweichen, da sich diese durch Drehungen ja ändern können; es wird nur auf gleiches Vorkommen von X und Y geachtet. Stimmen die Signaturen nicht überein, sind sie nicht in einander überführbar.

Nun wird sich parallel in beiden Bäumen, von den Blättern bis zur Wurzel hochgearbeitet:

- Blätter sind immer von Baum A zu Baum B überführbar.
- Für jeden inneren Knoten wird geprüft, dass die Vorzeichen seiner Signatur zwischen den zwei Bäumen entweder exakt identisch, für alle X umgekehrt, für alle Y umgekehrt, oder für alle Verbindungen umgekehrt sind. Diese Vorzeichenregeln gelten auch innerhalb von Superknoten.

Ein Beispiel:

Baum A hat die Signatur

$[(+Y, [(-X, "Blatt")]), (-Y, [(-X, "Blatt")]), (+X, [S(+Y, +X, -Y, -X);])]$.

Baum B hat die Signatur

$[(+Y, [(+X, "Blatt")]), (-Y, [(-X, "Blatt")]), (+X, [S(-Y, +X, +Y, -X);])]$.

Die Signaturen sind in ihren Abfolgen der Achsen X und Y identisch. Der erste Knoten $[(-X, "Blatt")]$ wurde um die Y -Achse gedreht, wodurch sich das Vorzeichen von X geändert hat. Der Zyklus S wurde um die X -Achse gedreht, wodurch sich die Vorzeichen aller internen Y -Verbindungen geändert haben.

Wenn auch diese Überprüfung gelingt, bedeutet dies, dass die Figuren in einander überführbar sind. Diese Überprüfung allein würde für die Lösung des Problems genügen, jedoch können durch die vorherigen Überprüfungen einige trivialen Negativbeispiele früh erkannt werden.

3 Laufzeit

Für V Plättchen mit E Verbindungen ist die Erstellung des Block-Cut-Trees über DFS $O(V + E)$. Die Suche der Zentren ist annähernd $O(V)$. Das Generieren der Signaturen ist aufgrund der Sortierung der Listen mit je maximal 4 Elementen unter $O(V \cdot \log(4))$ ⁴ und damit $O(V)$. Der Lösungsansatz ist also auch für größere V und E effizient.

4 Werkzeuge

Der Code wurde vollständig in Rust 1.94.1 (e408947bf 2026-03-25) geschrieben. Das `petgraph` Crate wurde für eine optimierte Implementierung von Graphen verwendet. Es verwaltet die Lifetimes von Knoten und Kanten intern, was einigen Boilerplate-Code erspart.

5 Beispiele

```
1
2
3
```

6 Quellcode

Auslassungen sind hier mit `/* [...] */` markiert.

```
fn solve() {
    return;
}
```

⁴https://doc.rust-lang.org/std/primitive.slice.html#method.sort_by_key